

UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Localización de estaciones de carga para vehículos eléctricos con tipos de estación y capacidad de subestación: Algoritmo Genético y Enfriamiento Simulado

Carlos Broch Cubedo
Máster Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital
Técnicas Metaheurísticas – DSIC-UPV

Curso 2026–2027

Índice

1. Descripción del problema **3**
1.1. Formulación formal . . . 3
1.2. Tipo del problema . . . 4
1.3. Ejemplo . . . 4

2. Algoritmo Genético (AG) **5**
2.1. Descripción del método aplicado. Diseño del algoritmo y parametrización . . . 5
2.1.1. Diseño del individuo: codificación y decodificación . . . 5
2.1.2. Función objetivo (fitness) . . . 5
2.1.3. Generación de la población inicial . . . 5
2.1.4. Tratamiento de individuos inválidos . . . 5
2.1.5. Selección, cruce, mutación y reemplazo . . . 6
2.1.6. Parámetros del AG . . . 6
2.2. Implementación de la solución . . . 7
2.2.1. Lenguaje y librerías . . . 7
2.2.2. Implementación . . . 7
2.3. Evaluación . . . 7
2.3.1. Casos de prueba y diseño experimental . . . 7
2.3.2. Variación de los parámetros del AG . . . 8
2.3.3. Experimentación con la configuración recomendada . . . 10

3. Enfriamiento simulado (ES) **13**
3.1. Descripción del método aplicado. Diseño del algoritmo y parametrización . . . 13
3.1.1. Diseño de la solución: codificación y decodificación . . . 13
3.1.2. Función objetivo (fitness) y escala de Δ . . . 13
3.1.3. Solución inicial . . . 13
3.1.4. Tratamiento de soluciones inválidas . . . 13
3.1.5. Vecindad . . . 14
3.1.6. Enfriamiento y criterio de parada . . . 14
3.1.7. Parámetros del ES . . . 14
3.2. Implementación de la solución . . . 15
3.2.1. Lenguaje y librerías . . . 15
3.2.2. Implementación . . . 15
3.3. Evaluación . . . 15
3.3.1. Casos de prueba y diseño experimental . . . 15
3.3.2. Variación de los parámetros del ES . . . 16
3.3.3. Experimentación con la configuración recomendada . . . 20
3.3.4. Comparación con el algoritmo genético . . . 21

4. Conclusiones **23**

1. Descripción del problema

El problema consiste en decidir en qué nodos de una ciudad se instalan estaciones de carga para vehículos eléctricos y de qué tipo. La ciudad se representa como un grafo $G = (V, E)$ cuyos nodos son intersecciones y cuyas aristas son tramos de calle con longitud l_e. Solo un subconjunto $J \subseteq V$ de nodos es candidato a albergar una estación. En cada candidato puede instalarse una estación *lenta* (más barata, menor capacidad y menor radio de cobertura) o una *rápida* (más cara, mayor capacidad y mayor radio), o ninguna. Cada candidato j está conectado a una subestación eléctica $g(j) \in K$, tal que la capacidad de la subestación limita de forma directa la construcción en masa por zonas.

El objetivo del problema es maximizar la demanda total de la ciudad atendida de forma efectiva. Pero para cumplir dicho objetivo es necesario respetar una serie de restricciones como son, no superar el presupuesto, no superar la capacidad de cada estación de carga instalada, una zona solo puede ser atendida por una estación situada dentro de su radio de cobertura y no superar la capacidad eléctrica de ninguna subestación.

1.1. Formulación formal

Conjuntos.

- V (nodos)
- E (aristas)
- $J \subseteq V$ (candidatos)
- $T = \{\text{lenta}, \text{rápida}\}$ (tipos de estación)
- K (subestaciones, con $g(j) \in K$ la subestación del candidato j)

Parámetros.

- $d(i, j)$ es la distancia de camino más corto (Dijkstra)
- w_i la demanda del nodo $i \in V$
- c_j^t el coste de instalar una estación de tipo t en j
- cap_j^t es la capacidad de la estación
- R^t es el radio de cobertura de la estación ($R^{rápida} > R^{lenta}$)
- B el presupuesto total
- Cap_k es la capacidad de la subestación k

Variable de decisión. $x_j^t \in \{0, 1\}$ indica si se instala una estación de tipo t en el candidato j. E $y_{ij}^t \geq 0$ es la demanda del nodo i atendida por la estación de tipo t instalada en j, definida solo si $d(i, j) \leq R^t$.

Función objetivo. La idea es maximizar la demanda atendida en cada nodo del grafo por cada estación instalada.

$$\max_{x,y} \; F(x, y) = \sum_{i \in V} \sum_{j \in J} \sum_{t \in T} y_{ij}^t \tag{1}$$

Restricciones.

$$\sum_{t\in T} x_j^t \leq 1 \qquad \forall j \in J \tag{2}$$

$$\sum_{j\in J}\sum_{t\in T} c_j^t\, x_j^t \leq B \tag{3}$$

$$\sum_{i\in V} y_{ij}^t \leq cap_j^t\, x_j^t \qquad \forall j \in J,\ \forall t \in T \tag{4}$$

$$\sum_{j\in J}\sum_{t\in T} y_{ij}^t \leq w_i \qquad \forall i \in V \tag{5}$$

$$y_{ij}^t = 0 \quad \text{si } d_G(i,j) > R^t \tag{6}$$

$$\sum_{j\in J:\, g(j)=k}\ \sum_{t\in T} cap_j^t\, x_j^t \leq Cap_k \qquad \forall k \in K \tag{7}$$

$$y_{ij}^t \geq 0 \qquad \forall i \in V,\ j \in J,\ t \in T \tag{8}$$

1.2. Tipo del problema

La combinatoria reside en los posibles valores de x, hay $3^{|J|}$ configuraciones posibles (sin estación, lenta o rápida por candidato), a lo que se suma además las restricciones de presupuesto y de subestación. Se trata de una variante del problema de localización de instalaciones con capacidades y cobertura (*CMCLP*), de tipo *NP-HARD*, lo que justifica el uso de algorítmos metaheurísticos que veremos a continuación.

1.3. Ejemplo

La Figura 1 muestra un extracto de una solución posible para la distribución de estaciones de carga de coches eléctricos en el centro de Lisboa. Sobre el mapa se dibujan los nodos candidatos con estación instalada, diferenciando las estaciones lentas (naranjas) de las rápidas (rojas).

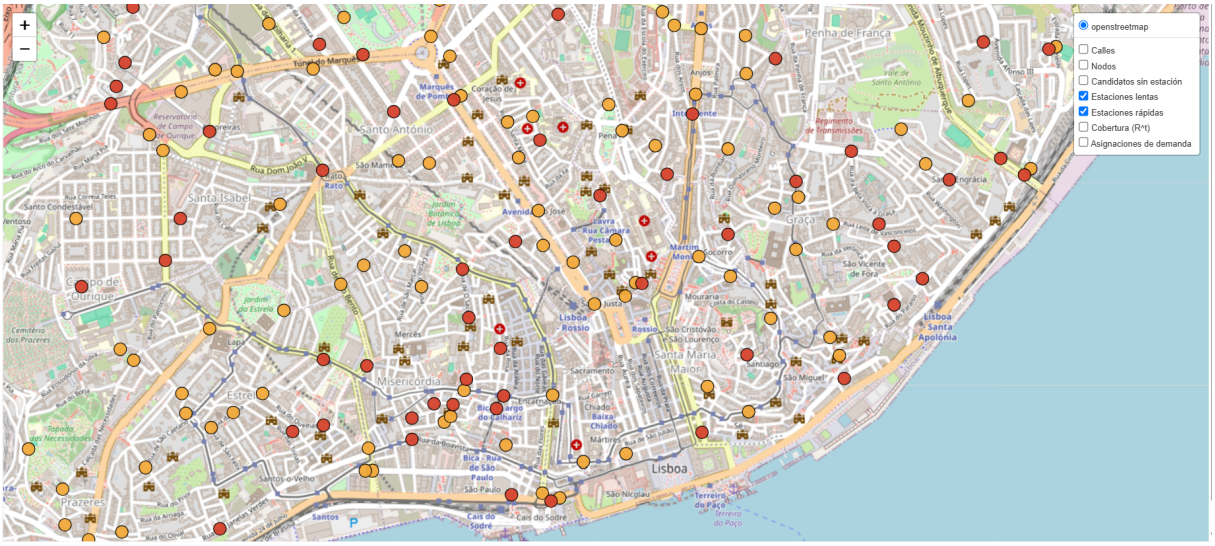


Figura 1: Extracto de la localización de estaciones de cargas en el centro de Lisboa.

Como se observa la demanda no es uniforme. Se generan centros de alta demanda de recarga, en estos puntos la solución muestra pequeñas agrupaciones de estaciones, con varias instalaciones muy próximas entre sí. Esto ocurre porque la capacidad de una sola estación se agota en estos puntos, y hacen falta varias cercanas para atender la demanda de los nodos de la zona.

Las estaciones rápidas tienen mayor capacidad y mayor radio de cobertura, pero también mayor coste. Con un presupuesto limitado, el algoritmo las reserva para donde más demanda pueden absorber. Mientras que, las estaciones lentas, más baratas, cubren la demanda dispersa que queda repartida.

Además, la capacidad de cada subestación eléctrica limita cuánta capacidad se puede concentrar en una misma zona, por lo que la solución debe repartir las estaciones en lugar de colocar solo rápidas en puntos calientes.

2. Algoritmo Genético (AG)

2.1. Descripción del método aplicado. Diseño del algoritmo y parametrización

2.1.1. Diseño del individuo: codificación y decodificación

Se ha optado por representar a cada individuo mediante un vector

$$\mathbf{x} = (x_1, \ldots, x_{|J|}), \qquad x_j \in \{0, 1, 2\}, \tag{9}$$

donde $x_j = 0$ significa que no se instala estación en j, $x_j = 1$ estación lenta y $x_j = 2$ estación rápida.

Esta codificación satisface por construcción la restricción (2) (un solo tipo por candidato), ya que cada gen solo puede tomar un valor.

Decodificación. A partir de $\mathbf{x}$ se obtiene la solución en dos pasos:

- Primero se obtiene x a partir de los genes, y comprobación de las restricciones (3) y (7) (presupuesto y subestaciones).
- Posteriormente, dado x, se reparte la demanda de los nodos entre las estaciones instaladas que los cubren ($d(i, j) \leq R^t$), respetando la capacidad de cada estación (4) y la demanda de cada nodo (5).

2.1.2. Función objetivo (fitness)

El fitness de un individuo factible es la demanda total atendida (1), calculada con la y obtenida en la decodificación:

$$f(\mathbf{s}) = \sum_{i \in V} \sum_{j \in J} \sum_{t \in T} y_{ij}^t. \tag{10}$$

Es un problema de maximización, por lo que los individuos con mayor f son mejores. Para una mayor compresión de los resultados se calcula el porcentaje de demanda atendida, $100 \cdot f / \sum_i w_i$.

2.1.3. Generación de la población inicial

La población inicial se genera de forma aleatoria, asignando a cada gen un valor de $\{0, 1, 2\}$. Como la generación uniforme suele romper algunas restricciones, como el presupuesto, los individuos iniciales se generan de modo que la mayoría de genes sean 0 con probabilidad baja de instalar estación. Si aún así resultan infactibles, se reparan mediante el procedimiento descrito en la siguiente sección 2.1.4.

2.1.4. Tratamiento de individuos inválidos

Los operadores de cruce y mutación pueden producir individuos que violan el presupuesto (3) o la capacidad de alguna subestación (7). Para remediar estos casos se ha optado por una

estrategia de reparación. Su funcionamientos se basa en eliminar o degradar el gen con peor relación de demanda que satisface/coste, de igual modo si es una violación de subestación. Así evitamos penalizar a los individuos en el fitness. Además, el coste de reparar individuos es asumible computacionalmente y no relentiza en exceso al algoritmo.

2.1.5. Selección, cruce, mutación y reemplazo

- **Selección.** Torneo de tamaño K. De esta forma disminuimos la presión de selección y permitimos a *malos* individuos tener más probabilidades de ser escogidos, de modo que se favorezca una mayor diversidad a través de las generaciones. A parte, en cada generación mantenemos a los n mejores individuos para realizar cruces con ellos.
- **Cruce.** Cruce por subestación entre dos padres con probabilidad p_c. Es un cruce que respeta la estructura, es decir, para cada subestación (*barrio*) se decide, al azar, si el hijo1 hereda todos los genes del bloque del padre1 o del padre2 (el hijo2 recibe el bloque contrario). Así se conservan configuraciones locales coherentes en lugar de mezclar genes sueltos de todo el mapa, como ocurre en el cruce uniforme. Si algún hijo no es factible se le repara.
- **Mutación.** En este caso aplicamos dos tipos de mutación con probabilidades distintas para generar mayor diversidad y exploración de soluciones. Primero partimos de la mutación *uniforme*, donde se altera cad gen con probabilidad p_m entre los valores $0, 1, 2$. A parte, contamos con la mutación *por intercambio* con probabilidad p_{mi}, en la cual se aplican m intercambios que pueden ser de *mover* (cierra una estación abierta y abre otra en un candidato cerrado, al azar), *tipo* (cambia una estación abierta de lenta a rápida o viceversa) y *abrir* (abre una estación nueva si aún cabe en presupuesto y subestación.). El algoritmo de mutación, en caso de no generar una solución factible, repite el movimiento hasta 10 veces con elecciones distintas, en caso contrario se descarta la mutación.
- **Reemplazo.** Generacional con elitismo, los e mejores individuos pasan directamente a la siguiente generación y el resto se sustituye por los descendientes generados por medio de la selección, cruce y mutación.

2.1.6. Parámetros del AG

La tabla 1 recoge los parámetros que permiten variar el rendimiento del algoritmo genético y sus posibles valores.

Tabla 1: Parámetros del Algoritmo Genético.

Parámetro	**Descripción**	**Dominio de los valores**
N	Tamaño de la población	$\mathbb{N}$
p_c	Probabilidad de cruce	$[0, 1]$
p_m	Probabilidad de mutación por gen	$[0, 1]$
p_{mi}	Probabilidad de mutación por intercambio	$[0, 1]$
m	Número de intercambios (mutación intercambio)	$\mathbb{N}$
k	Tamaño del torneo	$\mathbb{N}$
e	Número de individuos de élite	$\mathbb{N} \cup \{0\}$
$semillas_greedy$	Número individuos semilla tipo *greedy*	$\mathbb{N} \cup \{0\}$

2.2. Implementación de la solución

2.2.1. Lenguaje y librerías

El proyecto se ha implementado en Python, he programado el problema, el algoritmo genético y el enfriamiento simulado apoyándome en las siguientes librerías:

- `OSMnx` y `NetworkX`: obtención del grafo vial real desde OpenStreetMap y cálculo de caminos más cortos (Dijkstra).
- `NumPy`: representación vectorizada de individuos y población.
- `matplotlib`: gráficas del grafo y de la evolución del AG.
- `folium`: mapa real interactivo con el grafo y la solución.

2.2.2. Implementación

El programa se ejecuta desde `main.py`, que define la ciudad, los parámetros del problema y del cada algoritmo. Desde `main.py` se llama a otras funciones para la generación del problema, la incialización de los parámetros del algoritmo y su ejecución, además de otras funciones auxiliares para tratar resultados y mostrarlos al usuario.

Para dar realismo al estudio se emplean grafos de ciudades, barrios o pueblos reales obtenidos de OpenStreetMap. Inicialmente, se ha partido de grafos reducidos y sintéticos para depurar y probar el algoritmo, y se extrapola después a grafos basados en poblaciones reales.

Durante la ejecución se informa cada 25 generaciones del mejor fitness y de su mejora, y se guarda el historial para poder visualizar posteriormente la solución de la generación deseada. La parada puede configurarse por número de generaciones o por tiempo de cómputo. La solución final se dibuja sobre un mapa real, marcando todos los nodos con o sin estación y las aristas, y se generan gráficas de análisis como la evolución del fitness, tiempo, la utilización de recursos entre otras.

2.3. Evaluación

2.3.1. Casos de prueba y diseño experimental

Para evaluar la versatilidad del algoritmo genético (AG) se han definido ocho instancias que varían en la escala (de 100 a 2667 nodos) y la topología del grafo.

- **Serie sintética** (S-100, S-250, S-500 y S-1000): grafos geométricos aleatorios con grado medio constante y la misma densidad de nodos.
- **Serie real**: cuatro zonas descargadas de OpenStreetMap con `osmnx`: un pueblo (R-pueblo, Chinchón, radio de 600 m), un barrio en forma cuadrícula (R-grid, Eixample de Barcelona, 800 m), un centro histórico denso (R-centro, Madrid, Puerta del Sol, 1 km) y el centro de una ciudad grande (R-ciudad, Valencia, 2 km), con $|V| = 214$, 288, 719 y 2667 nodos respectivamente.

Instancia	Tipo	|V|	|E|	|J|	K	B	Demanda total
R-centro	real	719	1083	250	10	2102798	2288

Tabla 2: Ejemplo de instancia de prueba.

En todas las instancias se aplica la misma regla de generación de parámetros, $|J| = 35$ % de $|V|$ candidatos, K subestacioneseléctricas, presupuesto B igual al 20 % del coste de abrir todos los candidatos como estación rápida, radios de cobertura de 120 m (lenta) y 220 m (rápida) y una demanda concentrada en focos.

A continuación, cada instancia se ejecutada sobre distintas variaciones de parámetros donde se estudia su repercusión en la métrica a optimizar.

2.3.2. Variación de los parámetros del AG

Se ha estudiado el efecto de nueve parámetros variando uno cada vez respecto a una configuración de referencia (Tabla 2.3.2) sobre la instancia R-centro ($|V| = 719$, $|J| = 250$), con 15 semillas en total 23 configuraciones, y 345 ejecuciones. A parte, para evitar que las conclusiones vengan inducidas por la instancia se repitieron las pruebas sobre R-grid, R-pueblo y S-250.

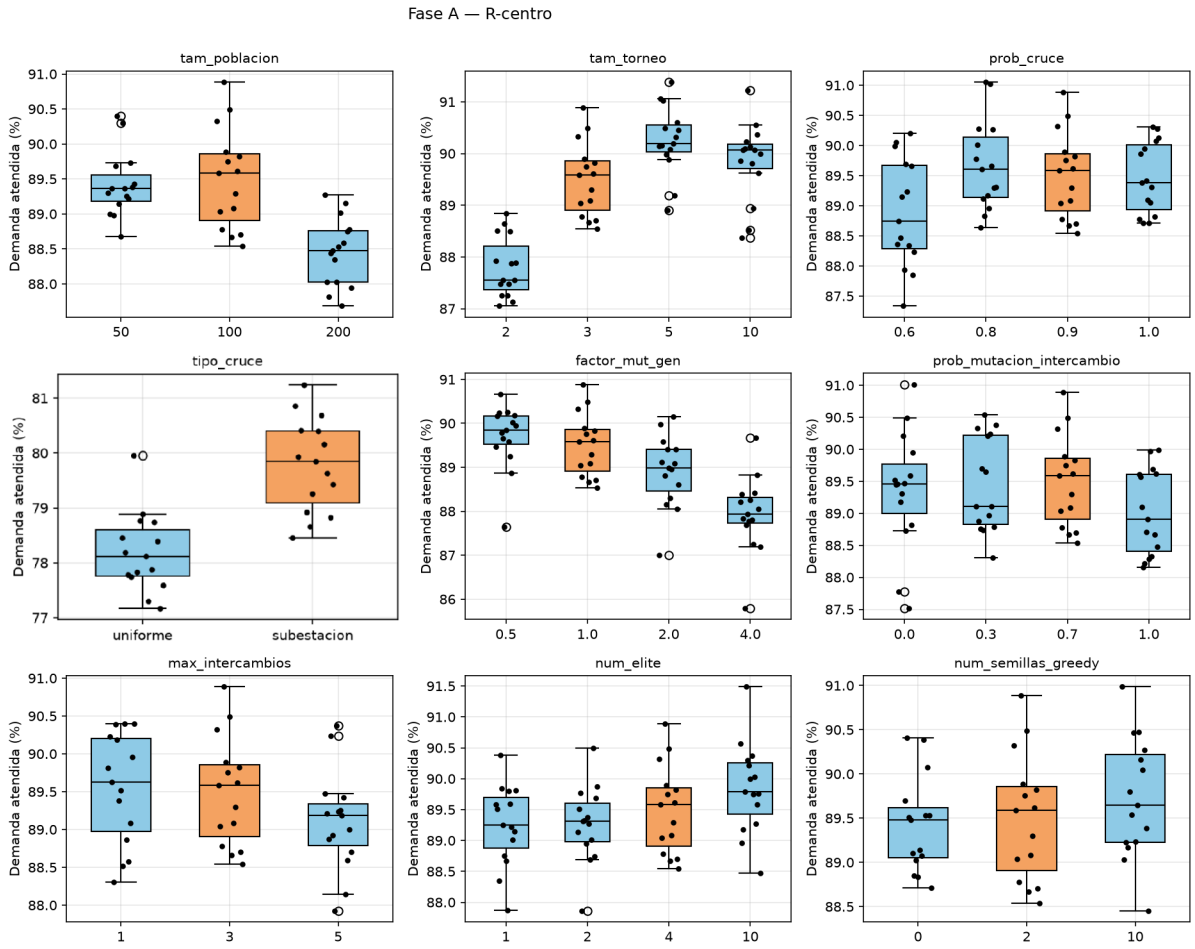


Figura 2: Demanda atendida (% de la demanda total) en R-centro para cada nivel de cada parámetro del AG, con 15 semillas por configuración. En naranja, el valor que tomamos de partida antes de empezar las pruebas y que suele estar entre la mitad de los valores probados. Los puntos son las ejecuciones individuales.

Estudiemos ahora la mejor elección de parámetros para el problema de la localización de estaciones de carga mediante algoritmo genético.

1. **Selección**. El tamaño del torneo es el parámetro con el mayor efecto sobre la demanda atendida. Con torneo 2 la presión selectiva es insuficiente y la demanda atendida cae. Con torneo 5 mejora, y con torneo 10 no mejora a 5, lo que sugiere que aumentar más la presión empieza a sacrificar la posible exploración. El torneo 5 es el mejor nivel en las cuatro instancias (Figura 4), con ganancias cercanas al 1 % sobre el torneo 3.
2. **Tamaño de la población.** La población de 200 individuos en la instancia R-centro, R-grid y R-pueblo es claramente peor que la 100, mientras que en S-250 ocurre lo contrario,

200 es el mejor nivel y el menos variable. Ésto nos dice que el valor óptimo dependerá del problema, pero 100 es el más robusto, porque nunca es el peor nivel.

3. **Mutación**. Aumentar la probabilidad de mutación por gen empeora el resultado de forma monótona. Posiblemente, cada gen modificado altere el coste y que el proceso de reparación acabe descartando los cambios realizados sobre estaciones incompatibles.

   Por otro lado, la mutación de intercambio, en cambio, aporta una mejora entre no usarla y usarla. El mejor nivel varía con la instancia, pero el valor por defecto que hemos asignado genera un buen rendimiento. Y en cuanto al número máximo de movimientos por mutación, 1 y 3 son equivalentes y 5 suele empeorar.

4. **Cruce**. El cruce por subestación tiene ventaja frente al cruce uniforme en todas las instancias. Una explicación posible es que conservar completa la configuración de un barrio preserva soluciones locales coherentes que el cruce uniforme rompe.

5. **Élite e inicialización voraz.** Los efectos son pequeños, el AG no depende, de la heurística voraz, usar 0, 2 o 10 semillas voraces en la población iniciañ no produce diferencias apreciables.

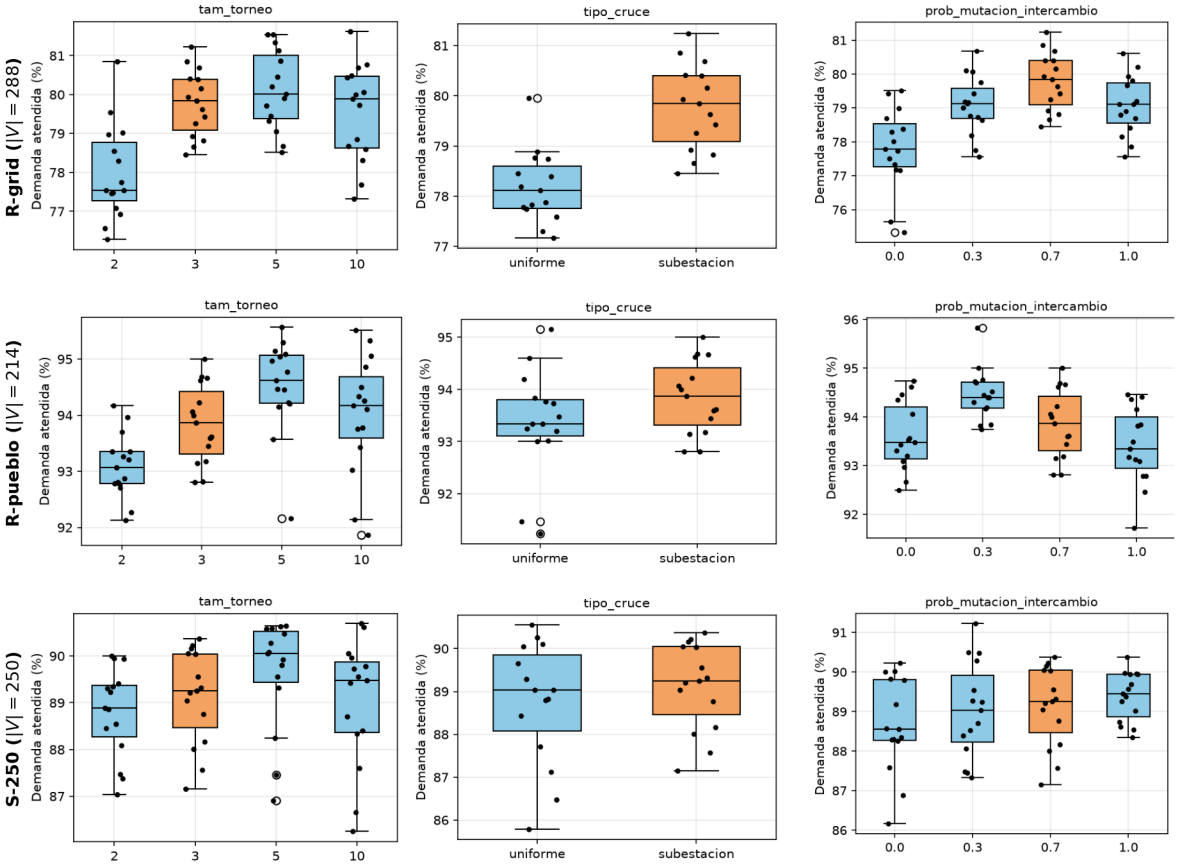


Figura 3: Pruebas sobre otras instancias (R-grid, R-pueblo y S-250): demanda atendida para el tamaño de torneo, el tipo de cruce y la probabilidad de mutación de intercambi.

La siguiente tabla reúne los niveles de los parámetros que sobresalen entre instancias, y por tanto, la configuración final de parámetros que usaremos sobre un problema real (R-ciudad) de gran talla.

Parámetro	Niveles probados	Final
Tamaño de población	50, 100, 200	100
Tamaño de torneo	2, 3, 5, 10	5
Probabilidad de cruce	0,6; 0,8; 0,9; 1,0	0,9 (0,8–1,0)
Tipo de cruce	uniforme, por subestación	subestación
Mutación por gen (múltiplo de $1/\|J\|$)	0,5; 1; 2; 4	0,5(0,5–1,0)
Prob. mutación de intercambio	0; 0,3; 0,7; 1	0,7 (0,3–0,7)
Movimientos máx. por intercambio	1, 3, 5	3 (1–3)
Individuos de élite	1, 2, 4, 10	4 (hasta 10)
Semillas voraces iniciales	0, 2, 10	2 (0–2)

Tabla 3: Parámetros del AG estudiados, niveles probados, valor de referencia y configuración recomendada (o, el rango de valores sin a penas diferencias).

2.3.3. Experimentación con la configuración recomendada

Con una misma configuración del AG para todas las instancias se ha estudiado cómo cambian la calidad y el coste con el tamaño y la morfología del problema. La Tabla 4 resume los resultados e incluye la comparación con la heurística voraz y la Figura 5 muestra la convergencia.

Instancia	$\|V\|$	Greedy (%)	AG (%)	Soluciones	Tiempo (s)	Lentas/Rápidas	Presup. (%)	Subest. (%)
S-100	100	73.1	96.4 ± 0.3	3543	45	9/4	99	71
R-pueblo	214	70.2	94.6 ± 0.9	8382	124	12/12	100	89
S-250	250	65.2	89.7 ± 0.7	7415	134	16/13	100	90
R-grid	288	63.0	80.3 ± 0.6	9412	126	12/17	100	90
S-500	500	64.3	78.7 ± 0.2	7934	378	32/25	100	86
R-centro	719	62.5	90.4 ± 0.7	10590	520	47/35	100	86
S-1000	1000	71.5	85.2 ± 0.3	8030	852	57/53	100	90
R-ciudad	2667	68.5	85.9 ± 0.4	13905	2382	118/159	100	86

Tabla 4: Resultados del AG con la configuración final y comparación con el greedy (% de la demanda total atendida).

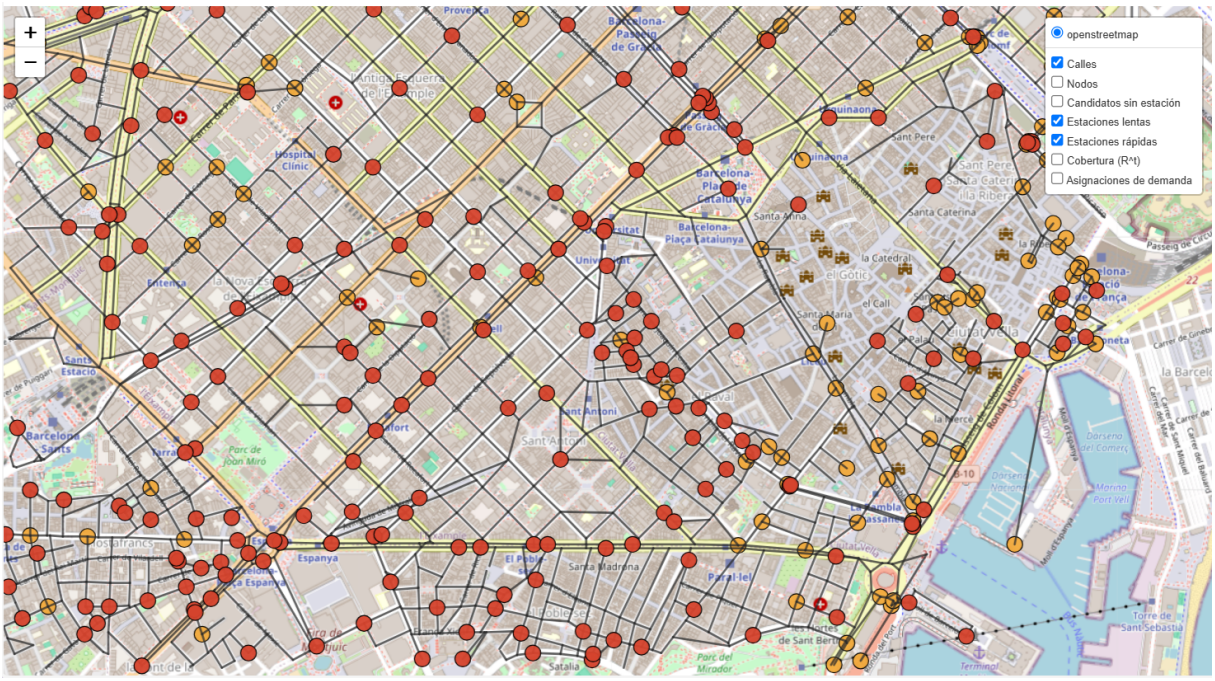


Figura 4: Visualización sobre mapa de la localización de estaciones con la configuración recomendada en L'Eixample en Barcelona.

Este mapa muestra la solución obtenida del algoritmo genético con los parámetros ajustados sobre L'Eixample, un grafo con 2457 nodos y 3873 aristas. En comparación con los resultados de L'Eixample (R-grid) (Figura 5) durante la fase de optimización de parámetros, se han mejorado los resultados de un 80 % de demanda atendida a un 97.8 %.

De los 2457 candidatos solo se instalan 721 estaciones (437 rápidas y 284 lentas). Se concentran en los focos de alta demanda, donde una sola estación agota su capacidad y hacen falta varias próximas entre sí, lo que explica las agrupaciones de la figura. Las rápidas son mayoría porque absorben más demanda y cubren más nodos cada una, y las lentas completan la cobertura en zonas de demanda dispersa.

En cuanto a las tallas de los problemas y su convergencia,

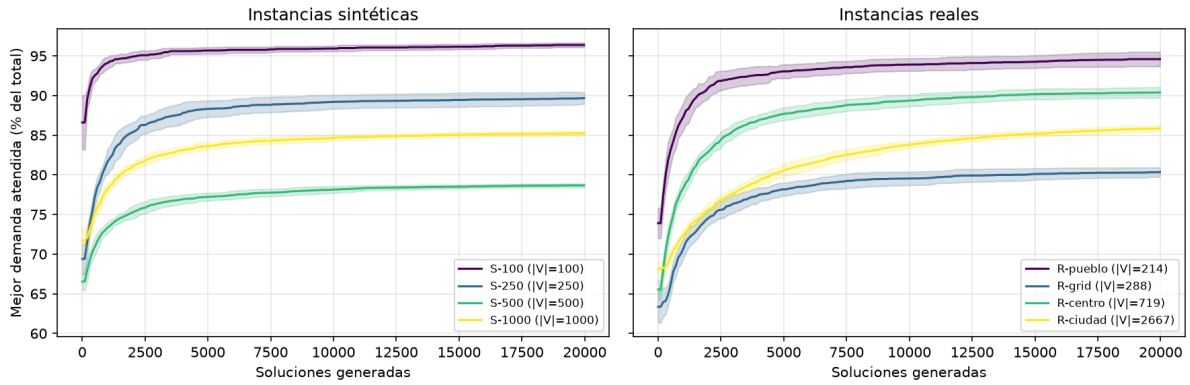


Figura 5: Mejor demanda atendida frente a soluciones generadas en las instancias sintéticas (izquierda) y reales (derecha).

Todas las curvas siguen el mismo patrón, un salto muy rápido en las primeras 2500 soluciones generadas y después una mejora cada vez más lenta, hasta casi aplanarse. El tamaño de la instancia influye en la velocidad de convergencia y en el número de soluciones generadas para converger. En las instancias pequeñas y medianas con 5000 soluciones se alcanza un fitness cercano al final, pero en las instancias de mayor talla no son suficientes y es necesario seguir evaluando nuevas soluciones.

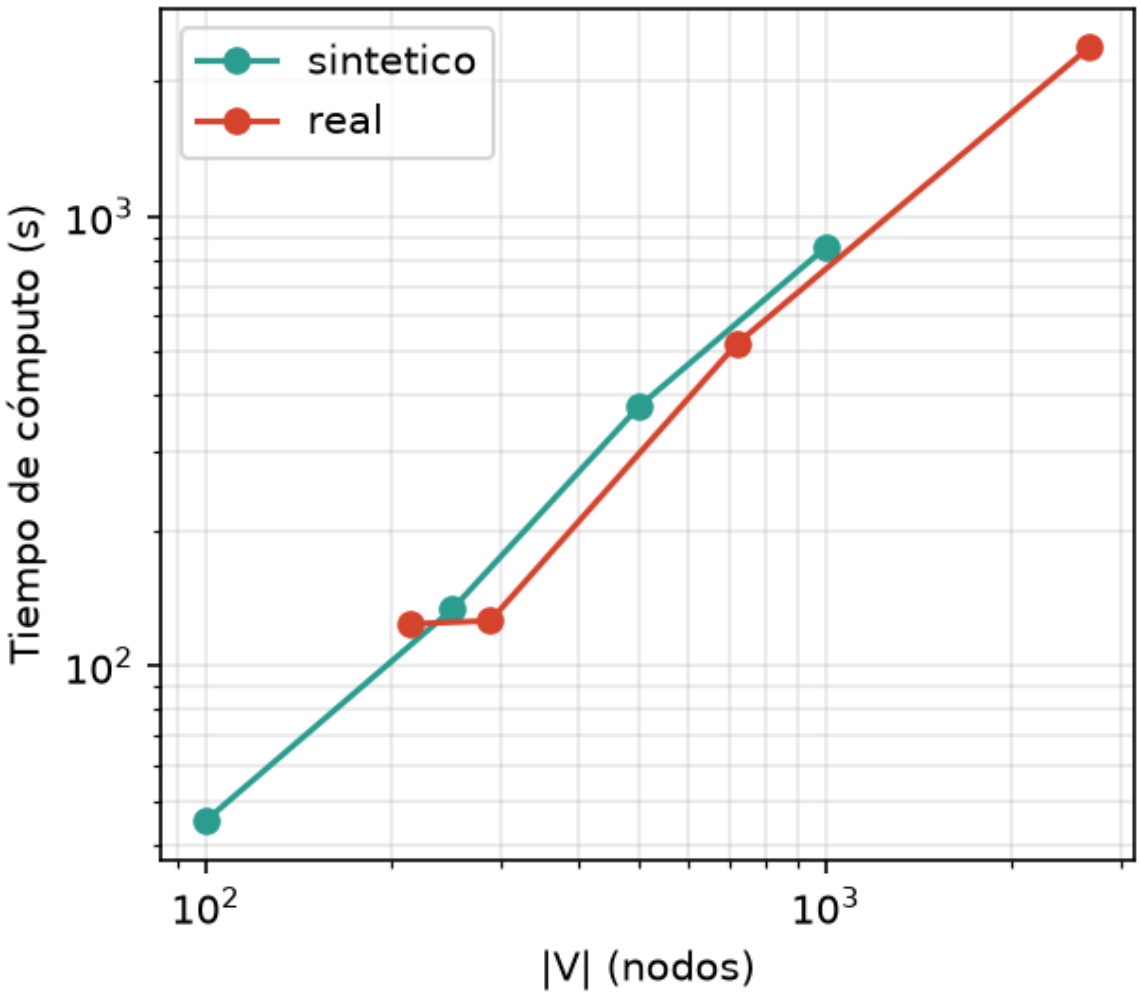


Figura 6: Efecto del tamaño del grafo en el tiempo de cómputo.

El tiempo parece seguir una tendencia lineal con el tamaño, de forma parecida en los dos tipos de problemas: de unos 45 s (S-100) a unos 850 s (S-1000), y de unos 125 s (R-pueblo) a unos 2400 s (R-ciudad).Este coste temporal muestra que el coste de evaluar un individuo aumenta con la talla del problema y descarta el hecho de aumentar el número de nodos escale de forma no lineal.

3. Enfriamiento simulado (ES)

3.1. Descripción del método aplicado. Diseño del algoritmo y parametrización

El enfriamiento simulado, a diferencia del algoritmo genético, trabaja con una única solución. En cada iteración se genera un vecino s' de la solución actual s y se decide si se acepta. Si s' no empeora a s se acepta siempre, y si la empeora se acepta con probabilidad

$$P(\text{aceptar } s') = e^{\Delta/T}, \qquad \Delta = f(s') - f(s) < 0, \tag{11}$$

donde f es el fitness (demanda atendida) y T es la temperatura. Al inicio T es alta y casi cualquier movimiento se acepta, lo que favorece la exploración del espacio de búsqueda. A medida que T desciende, los empeoramientos se aceptan cada vez menos y la búsqueda se vuelve progresivamente voraz. Al final de la ejecución la temperatura es tan baja que solo se aceptan mejoras.

3.1.1. Diseño de la solución: codificación y decodificación

Se utiliza exactamente la misma codificación que en el algoritmo genético: un vector $x = (x_1, \ldots, x_{|J|})$ con $x_j \in \{0, 1, 2\}$ (sin estación, lenta o rápida). Por construcción satisface la restricción (2), y la decodificación a las variables x_j^t y el posterior reparto de la demanda y_{ij}^t son idénticos a los del AG (apartado 2.1.1). Reutilizar la representación permite comparar ambos algoritmos en igualdad de condiciones.

3.1.2. Función objetivo (fitness) y escala de Δ

El fitness es el mismo del AG, la demanda total atendida (10). Como la probabilidad de aceptación (11) depende del cociente Δ/T, la temperatura se expresa en las mismas unidades que el fitness, es decir, en unidades de demanda atendida. Para interpretar un valor de T conviene compararlo con la demanda total $D = \sum_{i \in V} w_i$ de la instancia. En la instancia de evaluación ($D = 26\,136$), $T = 250$ equivale al 0,77 % de D. Por tanto, T_0 depende de la escala de la instancia. Equivalentemente, puede trabajarse con Δ normalizado dividiendo por D, con $T_0^{\text{norm}} = T_0/D$.

3.1.3. Solución inicial

El ES parte de una única solución, y se han considerado tres formas de obtenerla, que se comparan en el apartado de evaluación:

- **Aleatoria:** generada con el mismo procedimiento que los individuos de la población inicial del AG, con sesgo hacia no instalar estación y reparada si resulta infactible.
- **Voraz:** construida con la heurística voraz del AG, que ordena los candidatos por su ratio capacidad/coste y los abre mientras quepan en el presupuesto y en su subestación.
- **Solución del AG:** el mejor individuo de una ejecución previa del algoritmo genético. En este caso las evaluaciones consumidas por el AG se contabilizan como parte del coste.

3.1.4. Tratamiento de soluciones inválidas

Igual que en el AG, se ha optado por una estrategia de reparación. Si un vecino viola el presupuesto (3) o la capacidad de alguna subestación (7), se aplica el mismo operador de reparación, que desactiva la estación con peor relación capacidad/coste entre las que contribuyen a la restricción violada. Si la reparación deshace por completo el movimiento y devuelve la solución actual, se descarta ese vecino y se genera otro, hasta un máximo de 10 reintentos. De este modo no se penaliza el fitness y todas las soluciones evaluadas son factibles.

3.1.5. Vecindad

Un vecino se obtiene aplicando un único movimiento a la solución actual. Se utilizan cuatro operadores, todos reversibles, que se eligen al azar con probabilidades configurables:

- **Cambio de estado:** abre una estación en un candidato cerrado (lenta o rápida al azar) o cierra una estación abierta.
- **Intercambio:** cierra una estación abierta y abre otra, de tipo aleatorio, en un candidato cerrado.
- **Cambio de tipo:** convierte una estación abierta de lenta a rápida o viceversa.
- **Desplazamiento:** mueve una estación abierta, conservando su tipo, a uno de los k candidatos más próximos (según la distancia de camino más corto) que esté cerrado. La relación de proximidad es simétrica, de modo que b es vecino de a si está entre los k más próximos de a o viceversa, para que el movimiento sea reversible.

Los operadores son complementarios, el cambio de estado es el único que modifica el número de estaciones, el intercambio y el desplazamiento reubican estaciones sin variar ese número, y el cambio de tipo ajusta la capacidad de las ya instaladas. Se permite además aplicar de 1 a m movimientos encadenados por vecino.

3.1.6. Enfriamiento y criterio de parada

La temperatura se actualiza cada $L = 1$ vecinos evaluados. Se han implementado cuatro esquemas de enfriamiento, con T nunca inferior a $T_{\text{mín}}$:

$$\text{Geométrico:} \quad T_{i+1} = \alpha\, T_i, \quad 0 < \alpha < 1, \tag{12}$$

$$\text{Lineal:} \quad T_{i+1} = T_i - \delta, \tag{13}$$

$$\text{Inverso:} \quad T_{i+1} = \frac{T_i}{1 + \beta\, T_i}, \tag{14}$$

$$\text{Escalonado:} \quad T_{i+1} = \alpha\, T_i \text{ solo cada } s \text{ pasos.} \tag{15}$$

Para comparar los esquemas entre sí, los parámetros α, δ y β no se fijan a mano, sino que se calculan para que la temperatura descienda exactamente de T_0 a $T_{\text{mín}}$ a lo largo de todo el presupuesto de $N = (E_{\text{máx}} - 1)/L$ pasos de temperatura, siendo $E_{\text{máx}}$ el número máximo de evaluaciones:

$$\alpha = \left(\frac{T_{\text{mín}}}{T_0}\right)^{1/N}, \qquad \delta = \frac{T_0 - T_{\text{mín}}}{N}, \qquad \beta = \frac{1/T_{\text{mín}} - 1/T_0}{N}. \tag{16}$$

En cambio, en el calibrado de T_0 y α del apartado de evaluación, α se fija explícitamente y es uno de los parámetros que se estudian. En ese caso, con el esquema geométrico, la temperatura alcanza $T_{\text{mín}}$ tras

$$n = \frac{\ln(T_0/T_{\text{mín}})}{\ln(1/\alpha)} \text{ pasos de temperatura,} \tag{17}$$

es decir, n evaluaciones, a partir de ahí la búsqueda es voraz. Esta cantidad indica qué parte del presupuesto se dedica a explorar.

Una iteración del ES equivale a un vecino evaluado, por lo que el contador de evaluaciones del ES es directamente comparable con el de soluciones generadas del AG. Los criterios de parada disponibles son el número de evaluaciones, el tiempo de cómputo, el estancamiento (evaluaciones sin mejora del mejor fitness) y alcanzar $T_{\text{mín}}$.

3.1.7. Parámetros del ES

La tabla 5 recoge los parámetros que permiten variar el rendimiento del enfriamiento simulado y sus posibles valores.

Parámetro	Descripción	Dominio de los valores
T_0	Temperatura inicial (en unidades de demanda)	$\mathbb{R}^+$
$T_{\text{mín}}$	Temperatura mínima	$\mathbb{R}^+$, $T_{\text{mín}} < T_0$
Esquema	Tipo de enfriamiento	geométrico, lineal, inverso, escalonado
α (δ, β)	Factor de enfriamiento del esquema	$\alpha \in (0, 1)$
L	Vecinos evaluados por temperatura	$\mathbb{N}$
Solución inicial	Procedencia de la solución de partida	aleatoria, voraz, AG
Operadores	Probabilidad de cada operador de vecindad	$[0, 1]^4$
m	Movimientos máximos por vecino	$\mathbb{N}$
Parada	Criterio de parada	evaluaciones, tiempo, estancamiento, $T_{\text{mín}}$

Tabla 5: Parámetros del Enfriamiento Simulado.

3.2. Implementación de la solución

3.2.1. Lenguaje y librerías

El enfriamiento simulado se ha implementado en Python con las mismas librerías que el algoritmo genético (NetworkX, NumPy, matplotlib y folium para el mapa real). Reutiliza los módulos del AG sin modificarlos: la definición del problema, el cálculo de distancias, la codificación con su reparación y la función de fitness por flujo máximo.

3.2.2. Implementación

El algoritmo se encuentra en un módulo propio, que contiene la configuración, la vecindad, los esquemas de enfriamiento y el bucle principal. Un segundo programa `main_es.py`, análogo a `main.py`, construye la misma instancia que usa el AG, de modo que ambos algoritmos resuelven exactamente el mismo problema, y concentra los parámetros del ES. Un tercer programa `experimentos_es.py` lanza los experimentos de la evaluación (barrido conjunto de T_0 y α, esquemas, operadores, solución inicial y comparación con el AG) y guarda tablas y gráficas.

Durante la ejecución se guarda una muestra del historial cada 50 evaluaciones (mejor fitness, fitness de la solución actual, temperatura, tasa de aceptación y tiempo), y se informa periódicamente por consola. Con ese historial se generan gráficas de análisis propias del ES, como la convergencia, la evolución de la temperatura y la tasa de aceptación, además de la composición y el uso de recursos de la mejor solución. En el barrido se registra además, para cada ejecución, la temperatura final y la tasa de aceptación de vecinos peores en la ventana inicial, que permiten relacionar T_0 y α con el grado de exploración. La solución final se dibuja con las mismas funciones del AG.

3.3. Evaluación

3.3.1. Casos de prueba y diseño experimental

El calibrado de T_0 y α y la comparación con el AG se han realizado sobre una misma instancia objetivo real de gran tamaño, **[L'Eixample (Barcelona)]**, generada con las reglas del apartado 2.3.1. Los radios de cobertura son de 120 m (lenta) y 220 m (rápida). Cada configuración del calibrado se ha repetido con 5 semillas y 10 000 evaluaciones. La configuración de partida es el esquema geométrico con $L = 1$, $T_{\text{mín}} = 0{,}001$, los cuatro operadores de vecindad con un único movimiento por vecino, y solución inicial aleatoria.

Los estudios del esquema de enfriamiento, de la vecindad y de la solución inicial se realizaron antes, como estudio preliminar, sobre una instancia sintética ($|V| = 500$, $|E| = 3492$, $|J| = 175$,

$|K| = 8$, $B = 2\,906\,456$ y $D = 2\,831{,}1$) con 10 semillas, 20 000 evaluaciones y $T_0 = 200$. Sus conclusiones sobre el esquema y los operadores no dependen de la escala de la instancia.

3.3.2. Variación de los parámetros del ES

Temperatura inicial y factor de enfriamiento. Se ha barrido conjuntamente una rejilla de 5×5 combinaciones, con $T_0 \in \{10, 50, 100, 200, 10\,000\}$ y $\alpha \in \{0{,}8;\ 0{,}9;\ 0{,}95;\ 0{,}99;\ 0{,}999\}$, con esquema geométrico, $L = 1$ y 10 000 evaluaciones por ejecución. Según (17), la temperatura alcanza $T_{\text{mín}}$ tras entre 31 y 62 evaluaciones con $\alpha = 0{,}8$, entre 66 y 131 con $\alpha = 0{,}9$, entre 135 y 269 con $\alpha = 0{,}95$, entre 687 y 1 375 con $\alpha = 0{,}99$ y entre 6 904 y 13 809 con $\alpha = 0{,}999$ (según T_0), por lo que la rejilla cubre desde dedicar a explorar una fracción despreciable del presupuesto hasta no terminar de enfriar. La figura 7 y la tabla 6 muestran el fitness medio final, y la figura 8 las curvas de convergencia.

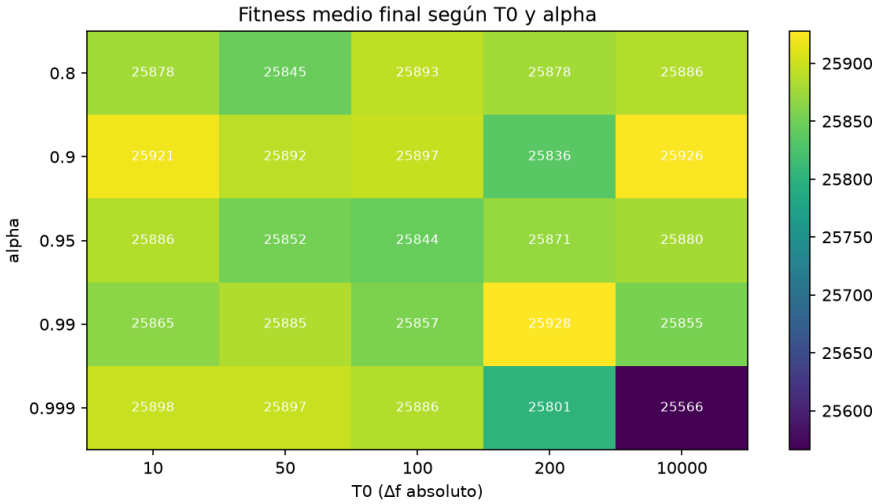


Figura 7: Fitness medio final según T_0 y α (esquema geométrico, 10 000 evaluaciones).

	$T_0 = 10$	$T_0 = 50$	$T_0 = 100$	$T_0 = 200$	$T_0 = 10\,000$
$\alpha = 0{,}8$	$25\,878 \pm 35$	$25\,845 \pm 44$	$25\,893 \pm 21$	$25\,878 \pm 53$	$25\,886 \pm 49$
$\alpha = 0{,}9$	$25\,921 \pm 29$	$25\,892 \pm 37$	$25\,897 \pm 55$	$25\,836 \pm 44$	$25\,926 \pm 22$
$\alpha = 0{,}95$	$25\,886 \pm 104$	$25\,852 \pm 82$	$25\,844 \pm 53$	$25\,871 \pm 38$	$25\,880 \pm 31$
$\alpha = 0{,}99$	$25\,865 \pm 21$	$25\,885 \pm 32$	$25\,857 \pm 82$	$\mathbf{25\,928 \pm 62}$	$25\,855 \pm 26$
$\alpha = 0{,}999$	$25\,898 \pm 43$	$25\,897 \pm 45$	$25\,886 \pm 34$	$25\,801 \pm 71$	$25\,566 \pm 69$

Tabla 6: Fitness medio final $\pm$ desviación típica entre semillas para cada combinación de T_0 y α (demanda atendida; $D = 26\,136$). En negrita, la combinación elegida.

La primera conclusión es que el resultado es poco sensible a ambos parámetros. Veintitrés de las 25 configuraciones quedan entre 25 836 y 25 928 de demanda atendida (98,85 % a 99,20 % de D). La diferencia entre la mejor y la peor de ellas, 92 unidades (0,35 % de D), es del orden de la desviación típica entre semillas (entre 21 y 104, con media 45), de modo que el barrido no permite afirmar que una de ellas sea mejor que otra. En particular, las configuraciones con enfriamiento rápido ($\alpha \leq 0{,}9$), que dedican como mucho unas 130 evaluaciones a temperaturas superiores a $T_{\text{mín}}$ y son por tanto prácticamente voraces, rinden similar que las que mantienen una fase de exploración más larga, y sus curvas de convergencia se superponen (figura 8).

Un efecto claro que se deduce es negativo y aparece al enfriar despacio partiendo de una

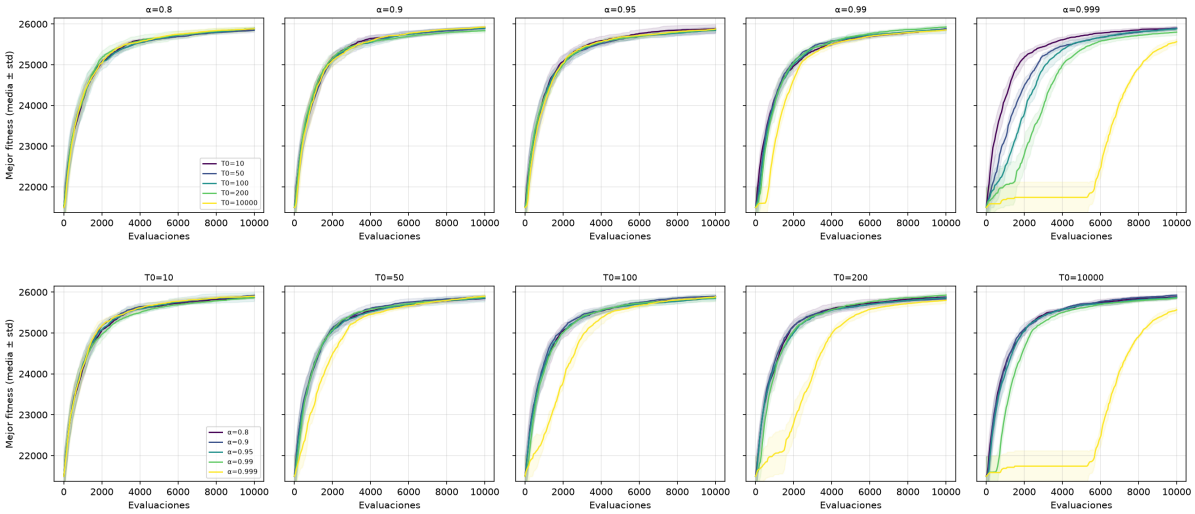


Figura 8: Mejor fitness frente a evaluaciones (media ± desviación típica entre semillas). Arriba, una gráfica por α con una curva por T_0; abajo, una gráfica por T_0 con una curva por α.

temperatura alta, porque entonces el presupuesto se consume explorando. Con $\alpha = 0,999$ y $T_0 = 200$ la temperatura no llega a $T_{\text{mín}}$ hasta cerca de la evaluación 9900. Con $\alpha = 0,999$ y $T_0 = 10\,000$ la temperatura ni siquiera llega a $T_{\text{mín}}$ y el resultado cae a 25 566, la curva permanece plana durante unas 5800 evaluaciones, en las que se aceptan el 99,8 % de los vecinos peores y la solución actual se comporta como un paseo aleatorio, y solo empieza a mejorar cuando T ha descendido bastante.

La tabla 7 muestra la tasa de aceptación de vecinos peores en la ventana inicial de cada ejecución, que permite ver cuánta exploración introduce cada combinación. Crece con T_0 y con α, desde el 8 % hasta el 99,8 %.

	$T_0 = 10$	$T_0 = 50$	$T_0 = 100$	$T_0 = 200$	$T_0 = 10\,000$
$\alpha = 0{,}8$	9,1	8,3	9,4	10,0	11,1
$\alpha = 0{,}9$	8,6	8,8	9,1	10,7	14,6
$\alpha = 0{,}95$	7,7	9,9	12,0	12,3	20,2
$\alpha = 0{,}99$	12,3	18,2	24,8	**44,6**	67,7
$\alpha = 0{,}999$	23,4	62,0	77,8	86,7	99,8

Tabla 7: Tasa de aceptación (%) de vecinos peores en la ventana inicial, media sobre semillas.

Se elige $T_0 = 200$ y $\alpha = 0,99$. Es la combinación de mayor fitness medio (el 99,20 % de D) y, conserva una fase de exploración moderada, con un 44,6 % de aceptación inicial de empeoramientos.

Esquema de enfriamiento. La figura 9 y la tabla 8 comparan los cuatro esquemas de enfriamiento con el mismo recorrido de temperatura desde T_0 hasta $T_{\text{mín}}$, en el estudio preliminar sobre la instancia sintética.

El esquema geométrico y el escalonado dan resultados prácticamente idénticos y son los mejores. El esquema inverso converge mucho más rápido, ya que $1/T$ crece linealmente y la temperatura cae enseguida a valores bajos, pero se estanca en un fitness inferior, lo que sugiere una convergencia prematura por falta de exploración. El esquema lineal es claramente el peor. Una posible explicación es que T permanece alta durante la mayor parte del presupuesto y solo desciende hacia el final, de modo que no queda tiempo suficiente para la fase de intensificación, y la curva sigue subiendo cuando se agotan las evaluaciones Es el mismo mecanismo que penaliza

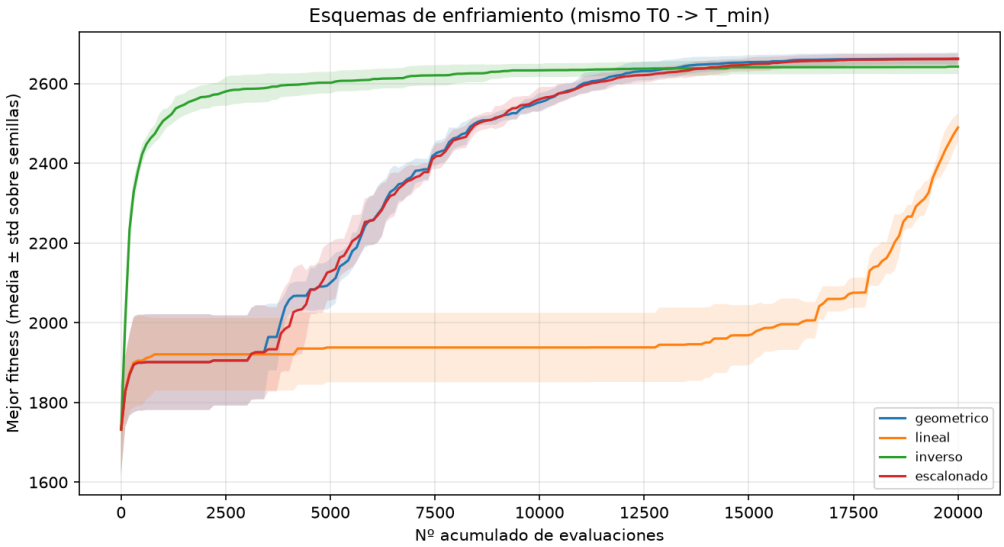


Figura 9: Mejor fitness frente a evaluaciones para los cuatro esquemas de enfriamiento (media ± desviación típica entre semillas). Estudio preliminar, instancia sintética.

Esquema	Fitness medio $\pm\,\sigma$	Mejor	Peor	Tiempo (s)
Geométrico	$2662{,}8 \pm 14{,}8$	2692,5	2646,3	549
Escalonado	$2661{,}5 \pm 14{,}3$	2692,5	2645,4	590
Inverso	$2642{,}7 \pm 18{,}6$	2673,4	2618,1	573
Lineal	$2489{,}8 \pm 34{,}4$	2543,6	2438,5	374

Tabla 8: Resultados de los esquemas de enfriamiento con 20 000 evaluaciones (estudio preliminar).

al enfriamiento lento con T_0 alto en el barrido anterior. Se escoge el esquema geométrico, que obtiene la mejor media con un único parámetro y es el más sencillo.

Vecindad. La figura 10 y la tabla 9 muestran el efecto de cada operador, usado en solitario, de la mezcla de operadores, y de los parámetros m (máximos movimientos de los operadores) y k (vecinos cercanos de un nodo), también en el estudio preliminar.

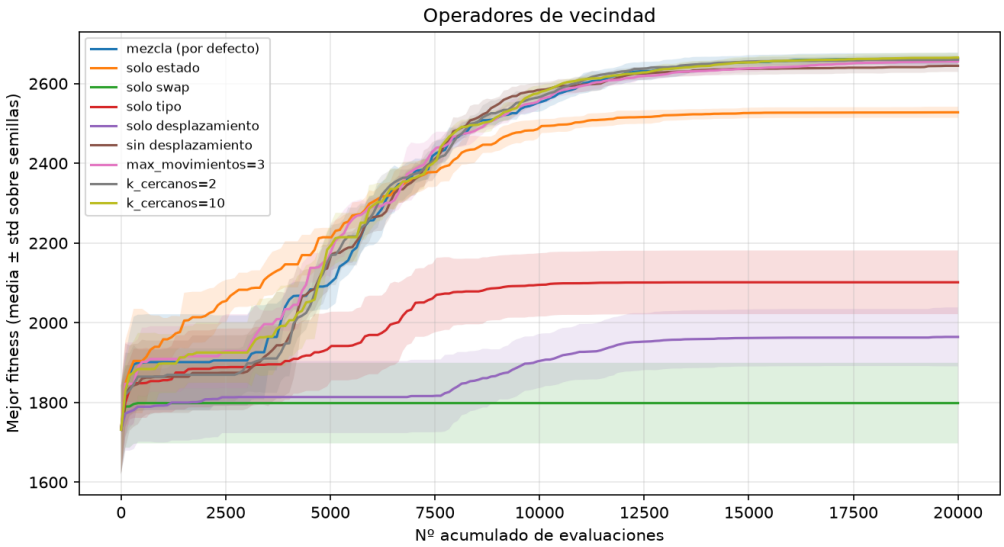


Figura 10: Mejor fitness frente a evaluaciones para distintas configuraciones de la vecindad (estudio preliminar).

Configuración	Fitness medio $\pm\,\sigma$
Mezcla (referencia)	$2662{,}8 \pm 14{,}8$
Sin desplazamiento	$2644{,}6 \pm 15{,}2$
$m = 3$ movimientos	$2655{,}5 \pm 11{,}1$
$k = 2$	$2659{,}9 \pm 10{,}4$
$k = 10$	$2665{,}4 \pm 11{,}3$
Solo cambio de estado	$2527{,}9 \pm 13{,}8$
Solo cambio de tipo	$2101{,}5 \pm 79{,}8$
Solo desplazamiento	$1964{,}5 \pm 73{,}8$
Solo intercambio	$1798{,}5 \pm 101{,}0$

Tabla 9: Fitness medio final para las configuraciones de la vecindad (estudio preliminar).

Ningún operador por sí solo es suficiente, y la mezcla supera en más de 130 puntos al mejor de ellos. El intercambio, el desplazamiento y el cambio de tipo no modifican el número de estaciones, por lo que la búsqueda queda limitada por la cantidad de estaciones de la solución inicial y se estanca en valores bajos con una variabilidad muy alta. El cambio de estado sí permite abrir y cerrar estaciones. Eliminar el desplazamiento empeora ligeramente el resultado, por lo que su aportación suele ser pequeña. Aplicar hasta 3 movimientos por vecino y variar k entre 2 y 10 no producen diferencias apreciables. Asimismo, evaluar distintos valores de vecinos por temperatura ($L \in \{1, 5, 25, 100\}$) ofrece un rendimiento prácticamente idéntico, y como al evaluar más vecinos incrementa el tiempo es preferible optar por valores entre 1 y 5. Se mantiene la mezcla de los cuatro operadores con un único movimiento por vecino, $k = 5$ y $L = 1$.

Solución inicial. La figura 11 compara partir de una solución aleatoria, voraz o del AG en la instancia sintética. Con 20 000 evaluaciones, el fitness medio final es prácticamente el mismo ($2662{,}8 \pm 14{,}8$, $2661{,}7 \pm 9{,}1$ y $2662{,}5 \pm 20{,}3$ respectivamente).

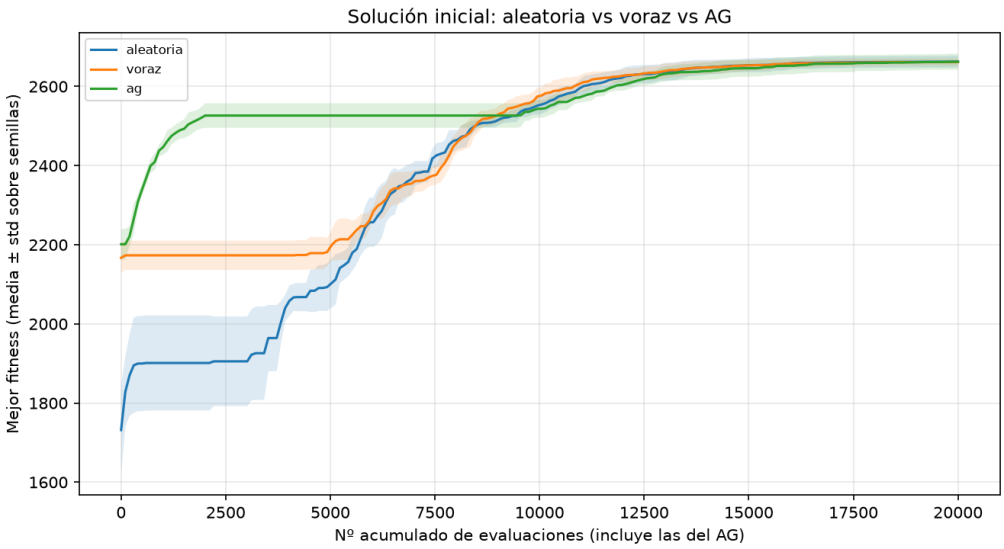


Figura 11: Mejor fitness frente a evaluaciones según la solución inicial (en el caso del AG se incluyen las evaluaciones del propio AG). Estudio preliminar.

Las curvas de la solución voraz y de la del AG permanecen planas hasta cerca de las 5 000 evaluaciones y después siguen la misma trayectoria que la aleatoria. Con una temperatura inicial alta la solución actual se aleja de la inicial al aceptar empeoramientos, y el mejor histórico no vuelve a mejorar hasta que T baja lo suficiente. Igualmente, aunque por culpa del proceso de exploración no se mejoren las soluciones voraces ni del algoritmo genético inicialmente, esta fase, donde aceptamos soluciones peores, es esencial para realizar una mejora local de calidad y poder escapar de convergencias prematuras o máximos locales inducidos por las soluciones de partida.

Configuración final. La tabla 10 resume los parámetros estudiados y la configuración final.

Parámetro	Niveles probados	Final
Temperatura inicial T_0	10; 50; 100; 200; 10 000	200
Factor de enfriamiento α	0,8; 0,9; 0,95; 0,99; 0,999	0,99
Esquema de enfriamiento	geométrico, escalonado, inverso, lineal	geométrico
Operadores	solos, sin desplazamiento, mezcla	mezcla
Movimientos máx. por vecino	1, 3	1 (1–3)
Vecinos evaluados por temperatura L	1, 5, 25, 100	1 (1–5)
Candidatos cercanos k	2, 5, 10	5 (2–10)
Solución inicial	aleatoria, voraz, AG	aleatoria

Tabla 10: Parámetros del ES estudiados, niveles probados y configuración recomendada. T_0 y α se han calibrado en la instancia objetivo, el resto, en el estudio preliminar.

3.3.3. Experimentación con la configuración recomendada

Con la configuración final se ha ejecutado el ES sobre la instancia objetivo con dos presupuestos. Con 10 000 evaluaciones (unos 50 minutos por ejecución) atiende el $(99{,}20 \pm 0{,}24)$ % de D, partiendo de soluciones aleatorias que atienden en torno al 75 %. Con 2 horas de cómputo (24 893 evaluaciones, una ejecución) llega al 99,73 % de D.

La evolución de la ejecución de 2 horas (figuras 12 y 13 del apartado siguiente) muestra tres fases ligadas a la temperatura. Con parada por tiempo, T pasa de 200 a 0,01 de forma geométrica a lo largo de las 2 horas (tabla 11). En la fase de exploración, el mejor fitness permanece en el valor de la solución inicial, a esa temperatura se acepta casi cualquier vecino y la solución actual deambula sin mejorar el mejor histórico. En la fase de descenso de la temperatura, el mejor sube del 84,5 % a cerca del 98 %. En la tercera fase, pasada la primera hora, y la búsqueda es prácticamente voraz y solo refina: 99,0 % a 1 h, 99,6 % a 1,5 h y 99,73 % a 2 h.

3.3.4. Comparación con el algoritmo genético

Se ha comparado el ES con el AG a igual tiempo de cómputo (2 horas) sobre la instancia objetivo de L'Eixample, con el ES en su configuración final. El AG usa la configuración del apartado 2.3.3 con una población de 1 000 individuos, con la que completa 30 generaciones en ese tiempo ($1\,000 + 30 \cdot 996 = 30\,880$ soluciones generadas, con élite de 4 individuos). La figura 12 muestra el fitness frente a las soluciones generadas, la figura 13 frente al tiempo de cómputo, y la tabla 11 recoge los resultados.

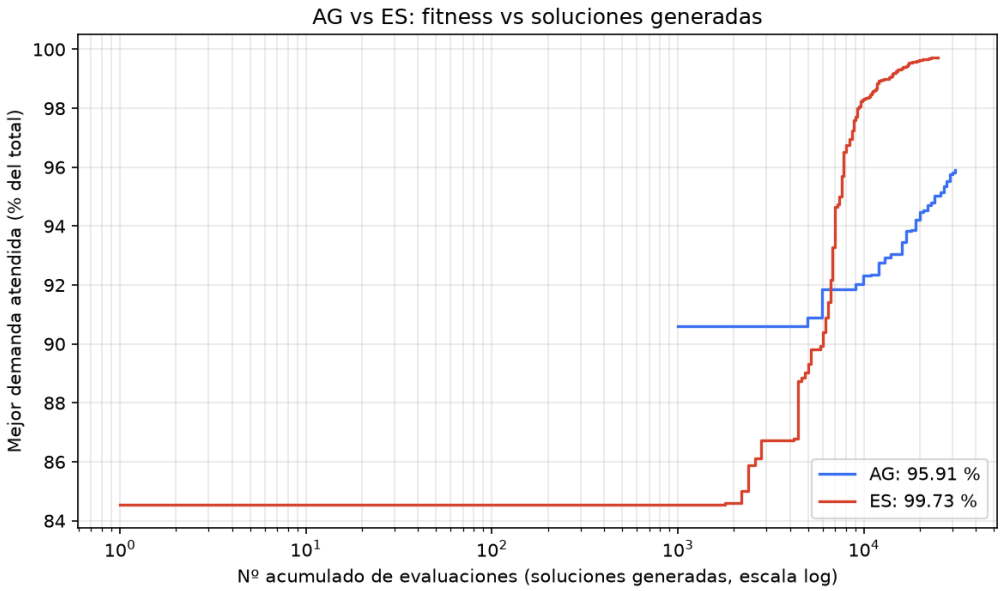


Figura 12: AG frente a ES: mejor demanda atendida (% de D) frente al número acumulado de soluciones generadas (escala logarítmica).

	Fitness final	% de D final	Soluciones generadas	Tiempo (h) total	% de D a las... 0,5 h	1 h	1,5 h	2 h
AG	25 067	95,91	30 880	2,04	91,85	93,05	94,80	95,82
ES	26 066	99,73	24 893	2,00	95,01	99,01	99,60	99,73
Temperatura del ES, $T(t)$					16,8	1,41	0,12	0,01

Tabla 11: Comparación entre el AG y el ES a igual tiempo de cómputo. El AG termina la generación en curso al cumplirse las 2 h (de ahí 2,04 h y la diferencia entre la última columna y el valor final).

Los dos algoritmos se comportan de forma distinta. El AG arranca mejor: tras evaluar su población inicial ya atiende el 90,6 % de la demanda, gracias a que esa población incluye soluciones voraces y al mejor de 1000 individuos, mientras que el ES parte de una solución aleatoria del

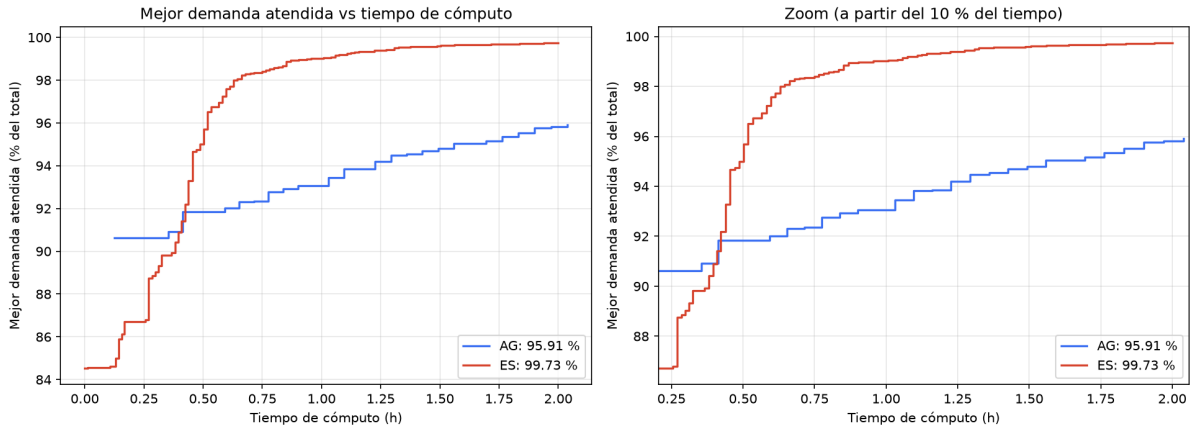


Figura 13: AG frente a ES: mejor demanda atendida (% de D) frente al tiempo de cómputo (izquierda) y ampliación a partir del 10 % del tiempo (derecha).

84,5 % y, por la fase de alta temperatura, no mejora su mejor histórico hasta cerca de las 1800 evaluaciones. Sin embargo, el ES adelanta al AG en torno a las 0,4 h (unas 5000–6000 soluciones generadas) y a partir de ahí se aleja. A 0,5 h lleva 3,2 puntos porcentuales de ventaja, a 1 h casi 6, y termina en el 99,73 % frente al 95,91 % del AG, es decir, unas 1000 unidades de demanda más. El ES prácticamente se satura tras la primera hora mientras que el AG sigue mejorando hasta el final, por lo que no puede decirse que haya convergido y no cabe descartar que con más tiempo redujera la diferencia.

Estos resultados deben leerse con cautela por tres limitaciones: se trata de una sola ejecución por algoritmo, de una sola instancia, y de una población del AG distinta de la recomendada. Con todo, la diferencia final (unas 1000 unidades) es un orden de magnitud mayor que la dispersión entre semillas medida para el ES en el calibrado y para el AG en la tabla 4 ($\pm$0,4 puntos para R-ciudad), lo que sugiere que la ventaja del ES con tiempos largos es real.

4. Conclusiones

En este trabajo se ha tratado la localización de estaciones de carga para vehículos eléctricos con dos tipos de estación (lenta y rápida) y restricciones de presupuesto y de capacidad eléctrica de subestación. El problema es una variante *NP-hard* del problema de localización con capacidades y cobertura, con $3^{|J|}$ configuraciones posibles. Se han resuelto instancias de hasta 2667 nodos con un algoritmo genético (AG) y un enfriamiento simulado (ES) que comparten codificación ($x_j \in \{0, 1, 2\}$), reparación y función de fitness, de modo que la comparación se hace en igualdad de condiciones.

En el AG, el tamaño del torneo fue el parámetro más influyente, y el valor 5 resultó el mejor en las cuatro instancias probadas. El cruce por subestación superó al uniforme en todas ellas, probablemente porque conserva configuraciones locales coherentes. La mutación por gen empeora al aumentar su probabilidad, mientras que la mutación por intercambio aporta una mejora clara. El tamaño de población, la élite y las semillas voraces tuvieron un efecto menor. Con una única configuración, el AG atendió entre el 78,7 % y el 96,4 % de la demanda en las ocho instancias, frente al 62–73 % de la heurística voraz. El tiempo de cómputo creció de forma aproximadamente lineal con el tamaño del grafo.

En el ES, el calibrado conjunto de la temperatura inicial y del factor de enfriamiento sobre la instancia objetivo mostró que el resultado es poco sensible a ambos mientras el enfriamiento no consuma todo el presupuesto: 23 de las 25 combinaciones atienden entre el 98,9 % y el 99,2 % de la demanda. Solo el enfriamiento lento con temperatura inicial alta ($\alpha = 0{,}999$ y $T_0 \geq 200$) empeora, porque gasta el presupuesto en una exploración casi aleatoria. Se adoptó $T_0 = 200$, $\alpha = 0{,}99$ y esquema geométrico, que dio el 99,2 % con 10 000 evaluaciones. En el estudio preliminar, ningún operador de vecindad resultó suficiente por sí solo, y la mezcla de los cuatro supera en más de 130 puntos al mejor operador individual.

A igual tiempo de cómputo (2 horas) sobre la instancia objetivo, el AG arranca mejor, pero el ES lo adelanta a los 25 minutos aproximadamente y termina claramente por encima (99,73 % frente a 95,91 % de la demanda), mientras que el AG sigue mejorando al final. La comparación se basa en una sola ejecución, una sola instancia y una población del AG de 1 000 individuos, por lo que debe confirmarse con varias semillas. Los datos sugieren que la ventaja del ES procede de la fase de exploración, y del refinamiento local con vecindad de un movimiento. En consecuencia, en esta instancia el ES es preferible salvo con presupuestos de tiempo muy cortos.